Claude Code for Accounting Research

Module 4 — WRDS MCP, Scale Discipline, and the Stata Bridge

John Barrios

Yale School of Management

Today's Arc

- **0:00–0:40** Lecture: MCP, architecture, iron laws, inspection, scale
- **0:40–1:15** Live demo: plain English to regression-ready funda extract
- **1:15–1:25** Break
- **1:25–2:45** Lab: two tracks — live WRDS or FSDS fallback
- **2:45–3:05** Debrief + exit ticket

Module 3 Ended With a Question

- CIKs identified, filings parsed. Next: attach leverage, profitability, and size
- Compustat and CRSP sit behind three barriers a chat tool cannot clear:
  - **Authentication** — a licensed, MFA-gated WRDS account
  - **Licensing** — your institution's subscription, not yours to redistribute
  - **Scale** — hundreds of variables, decades of history
- Today's stack handles all three, in order, on a real account

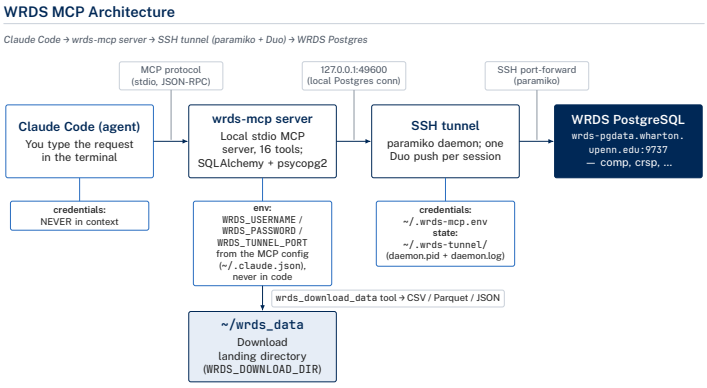
What an MCP Actually Is

- A small dedicated server handles connection, auth, and query submission; the agent talks to the server through a protocol, not to the service directly
- `wrds-mcp`: a local Python process exposing sixteen named tools (`wrds_run_sql`, `wrds_download_data`, CRSP/Compustat/CCM helpers…)
- Credentials are read from the environment, never from code or the conversation

Key point

The protocol handles auth, so your credentials never appear in the conversation.

WRDS MCP Architecture



Four hops: agent, `wrds-mcp` server, paramiko Duo tunnel, WRDS PostgreSQL. Credentials never enter Claude's context at any hop.

Governance Is Not Sandboxing

This applies well beyond WRDS

A sandbox constrains what an agent can *do*. It does nothing to constrain what an agent can *see* once data sits in front of it.

- What protects the password: it sits in an env file one link below where Claude reasons. Claude's good behavior is not the safeguard.
- Data governance is a separate, legal matter: the WRDS license is your institution's, and extracts are not yours to redistribute
- **Hard rule: zero WRDS data distribution.** Not in a lab zip, not on the site, not in Slack, ever

Ritual and Hygiene

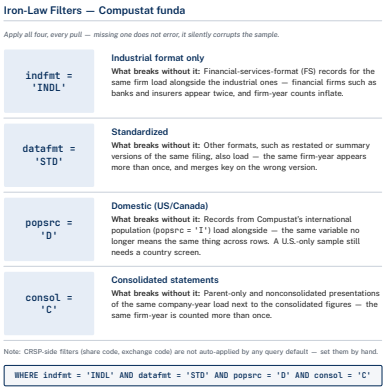
- Own virtual environment: `python3 -m venv ~/.wrds-mcp-env`, not system Python
- Credentials in `~/.wrds-mcp.env`, `chmod 600`; write the `.gitignore` entry *before* the file exists
- Tunnel ritual, strict order: **up** → **work** → **teardown**
  - `tunnel_up.sh`, approve the one Duo push, confirm port 49600 is listening
  - Query freely: one approved session covers the whole run
  - `tunnel_down.sh` at the end of every session

Common Failure

Pasting a password into the prompt “just this once”

Once a credential is typed into a conversation, it has been sent to the model. No cleanup afterward fully undoes that. The fix is procedural: credentials go in `~/.wrds-mcp.env`, full stop.

The Iron Laws



A query missing one of these runs without error and returns plausible-looking but wrong rows. The WRDS filter skill enforces all four.

CRSP: No Iron-Law Shortcut

- `wrds_get_crsp_returns` joins returns to names by permno and date range, and does nothing more
- Share-type / exchange restrictions are **not** applied automatically by any tool in this stack
- Identify which filters apply → validate before executing → execute → inspect. Every time, including for "a quick test"

Inspect Before You Trust

- Four mandatory checks, under a minute total: row count vs. expectation, `.head()`/`.sample()` eyeball, null check, date-range check
- "It ran without an error" $\neq$ "it returned the right rows"

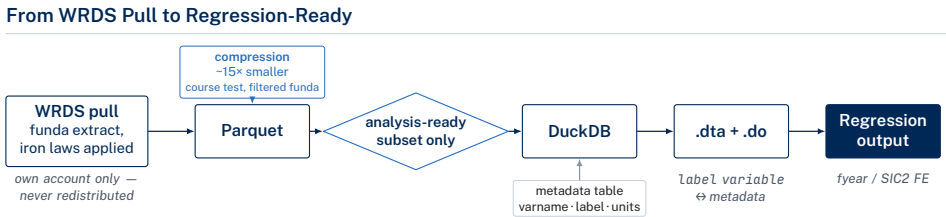
The Rationalization Table

What you tell yourself	Why it's a rationalization
"It ran without an error."	No error means valid syntax. It says nothing about correctness, filters, or joins.
"The row count looks about right."	"About right" is not a comparison with an expected number written down beforehand.
"I've pulled this table before, it's probably fine."	Prior experience doesn't verify *this* query, on *this* date range, with *these* filters.
"The query worked, so it's correct."	**The most dangerous one.** Treats "executed" and "correct" as one claim. They are two.

Scale Discipline

- **Filter at the database, not in pandas:** `WHERE` does the work before a row crosses the wire
- **Never** `SELECT *` on a WRDS table; name the columns you need
- Compustat's full annual file has hundreds of variables and decades of history. Never read it whole into a chat window or a DataFrame

Parquet → DuckDB → Stata



Filter at the source, store compressed (Parquet), query lazily (DuckDB), export only what's analysis-ready (`.dta`). In the course's own test, a filtered Compustat CSV extract shrank to roughly one-fifteenth its size as Parquet. You can reproduce this yourself; it is not a vendor claim.

Metadata as Context Engineering

- `dltt`, `che`, `csho`, `prcc_f`: opaque unless something on disk explains them
- Build once: a `compustat_metadata` table with varname, label, units, screening note
- Demo: close the session, open a fresh one, orient it with nothing but `SELECT * FROM metadata`

Module 1's mantra, at the data layer

Files persist; context doesn't. The table also survives handing the project to a coauthor who never saw your prompts.

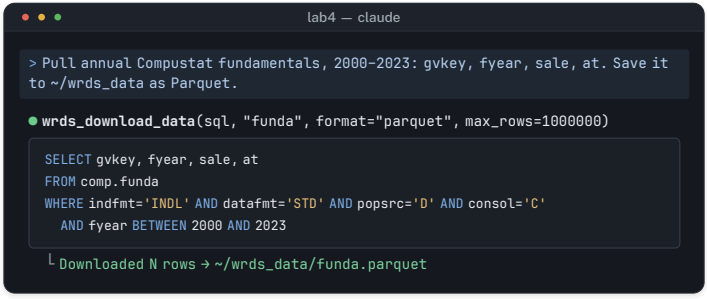
Feasibility-Assessment Prompting

Ask before you build. Use this exact wording:

> *"I want a firm-year panel with leverage and ROA for US industrials 2000--2023 from funda. Tell me what's involved before you try anything complicated. What might be missing?"*

Raises the filter question and linking issues *before* a single row is pulled. Use the prompt as written, not a paraphrase.

What You'll See: the Query



Stylized session transcript (not a screenshot).
Plain English in, the SQL wrds-mcp issues echoed back — no credentials, no data values.

A plain-English request, and the SQL `wrds_download_data` runs for you. The result lands in `~/wrds_data`, not in the conversation.

Sub-Agents: Delegating Work

- A separate Claude instance, its own fresh context, one bounded job
- Intermediate output stays inside it; only a written summary comes back
- Independent pieces can run in parallel: one per decade, industry, or filing batch
- Define your own as Markdown files in `.claude/agents/`; ask Claude to create or edit them

Ask in plain English:

> *"Use three sub-agents in parallel. Each one profiles one decade of funda.parquet (row counts, missing at and sale, duplicate gvkey-fyear pairs) and writes its results to checks/profile_<decade>.md. Then compare the three files."*

Delegation: What to Check

It does not see your conversation

It starts with only the task description. Filters, sample rules, and `fyear` conventions agreed in chat are invisible to it. Put the rules on disk and point it to them: "read `compustat_metadata` and `CLAUDE.md` first."

Its summary is a claim, not evidence

"No duplicates" arrives without the evidence behind it. Have it write the numbers to a file, and spot-check one yourself.

Rule of thumb: use sub-agents for independent pieces or output you don't need to read. Skip them for small, sequential steps.

Two Languages, One Answer

- Same FE regression, written twice (Python and Stata), each from the model description, not from the other script
- **Coefficients** should match to six decimals; a mismatch points to a wrong FE, a duplicating merge, or a flipped sign
- **Standard errors** can legitimately differ:
  - small-sample corrections
  - singleton handling (`reghdfe` drops singletons by default)
  - cluster degrees-of-freedom adjustments

The rule

Not everything has to match. Every gap must be explained by a named default you can point to.

Idea credit: Scott Cunningham, “Claude Code 31,” *Causal Inference* Substack, March 2026.

Next: Live Demo (0:40–1:15)

- Feasibility-assessment prompt on the instructor's own account (tunnel already up)
- Pull: funda extract, iron laws applied, explicit variable list, via wrds-mcp
- Inspect: all four checks, including one deliberate anomaly
- Land it: Parquet, DuckDB, metadata table, tested cold in a fresh session
- Teardown ritual, on camera

Next: The Lab — Two Tracks

Live track

- Checkpoint-cleared students
- venv → `.env` → tunnel up
- Feasibility prompt → funda pull, own account
- Inspect, Parquet, DuckDB, metadata
- **Tunnel teardown**

Fallback track

- Duo unresolved or tunnel failure
- Identical pipeline, public FSDS-derived sample shaped like funda
- Same filters (conceptual), same inspection
- Same Parquet, DuckDB, metadata steps

Both tracks teach the same procedure; only the data source differs. Extension (either track): the Stata bridge. Export an analysis-ready `.dta`, never full funda.

Verify Checklist

- Credential file outside the project; a `grep` for `$WRDS_PASSWORD` over the project returns nothing
- All four iron-law filters present (paste the `WHERE` clause)
- Four inspection checks written down with actual numbers
- Metadata table queryable; $\geq$8 variables documented
- Explicit variable list in every query on a WRDS table or raw extract — no `SELECT *`
- No WRDS-derived file outside `~/wrds_data` / project data dir; nothing committed to git

Last, on purpose

Tunnel torn down (live track). It is the last box on the list on purpose, so it is the last thing you check before you finish.

Debrief, Exit Ticket, and Homework

- Debrief: anomalies found during inspection (collect three from the room); which skipped iron law would have broken whose extract
- Fallback-track students: did the pipeline feel different? (It shouldn't.)
- Exit ticket: standard three + write the feasibility-assessment prompt for *your own* dissertation data pull